

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ



NHÓM 2

ỨNG DỤNG LẬP TRÌNH SONG SONG CUDA C++ TRONG XỬ LÝ VÀ PHÂN
TÍCH KHỐI DỮ LIỆU ẢNH Y KHOA 3D

Đề tài môn Lập Trình Song Song

Kỹ thuật điện tử và tin học

(Chương trình đào tạo chuẩn)

Hà Nội - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ



NHÓM 2

ỨNG DỤNG LẬP TRÌNH SONG SONG CUDA C++ TRONG XỬ LÝ VÀ PHÂN
TÍCH KHỐI DỮ LIỆU ẢNH Y KHOA 3D

Đề tài môn Lập Trình Song Song

Kỹ thuật điện tử và tin học

(Chương trình đào tạo chuẩn)

Giảng viên hướng dẫn: TS. Nguyễn Hoàng Trung

Hà Nội - 2026

Lời cảm ơn

Trước hết, chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến TS. Nguyễn Hoàng Trung người đã tận tình hướng dẫn, hỗ trợ và đồng hành cùng chúng em trong suốt quá trình học tập cũng như thực hiện báo cáo môn Lập trình song song.

Đề tài “Ứng dụng lập trình song song CUDA C++ trong xử lý và phân tích khối dữ liệu ảnh y khoa 3D” là cơ hội quý báu để chúng em có thể vận dụng những kiến thức đã học vào thực tế, đồng thời tiếp cận sâu hơn với lĩnh vực tính toán song song và xử lý ảnh y tế. Trong suốt quá trình thực hiện đề tài, chúng em đã nhận được rất nhiều sự chỉ bảo tận tình, những góp ý chuyên môn quý báu và sự động viên từ thầy. Đây không chỉ là nguồn hỗ trợ quan trọng giúp chúng em hoàn thành đề tài mà còn là nền tảng giúp chúng em nâng cao tư duy nghiên cứu và kỹ năng làm việc thực tế.

Thông qua báo cáo này, chúng em cũng nhận thức rõ rằng kiến thức và kinh nghiệm của bản thân vẫn còn nhiều hạn chế, vì vậy bài làm khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét từ thầy để có thể tiếp tục hoàn thiện bản thân cũng như nâng cao chất lượng nghiên cứu trong tương lai.

Cuối cùng, chúng em xin kính chúc thầy luôn mạnh khỏe, hạnh phúc và gặt hái nhiều thành công trong công tác giảng dạy cũng như nghiên cứu khoa học.

Mục lục

Lời cảm ơn	i
Danh sách hình vẽ	iv
Danh sách bảng	v
Danh sách tên viết tắt	vi
MỞ ĐẦU	1
1 TỔNG QUAN ĐỀ TÀI	3
2 CƠ SỞ LÝ THUYẾT	5
2.1 Tổng quan về khối dữ liệu 3D trong Y khoa	5
2.2 Kiến trúc Lập trình song song CUDA	5
2.3 Cơ sở thuật toán nhánh phân tích mạch máu (Vessel Analysis)	6
2.3.1 Thuật toán chiếu ảnh cường độ tối đa (MIP)	6
2.3.2 Bộ lọc trung bình (Mean Filter)	7
2.3.3 Thuật toán phân ngưỡng nhị phân (Thresholding)	8
2.3.4 Tính toán Mật độ mạch máu (Vessel Density)	9
2.4 Phân tích kết cấu mô bằng ma trận GLCM (Texture Analysis)	10
2.4.1 Độ tương phản (Contrast)	11
2.4.2 Độ đồng nhất (Homogeneity)	12
2.4.3 Năng lượng (Energy)	12
3 PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG	14
3.1 Kiến trúc hệ thống tổng quát (System Pipeline)	14
3.2 Tối ưu hóa I/O Dữ liệu (Data I/O Optimization)	14
3.3 Thiết kế và Triển khai các hàm Kernel CUDA	15
3.3.1 Kernel Chiếu ảnh cường độ tối đa (MIP_Projection)	15
3.3.2 Các Kernel thuộc nhánh Vessel Analysis	15
3.3.3 Kernel thuộc nhánh Texture Analysis (GLCM)	16
3.4 Quản lý bộ nhớ và Đồng bộ hóa	16
4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	18
4.1 Môi trường và Dữ liệu thực nghiệm	18
4.2 Đánh giá tốc độ và Hiệu năng tính toán (Performance)	18

4.3	Đánh giá kết quả trực quan (Định tính)	19
4.4	Đánh giá các chỉ số y khoa định lượng (Quantitative Metrics)	20
4.4.1	Nhánh phân tích hình học (Mạch máu)	20
4.4.2	Nhánh phân tích kết cấu (Texture – GLCM)	20
	Kết luận và Hướng phát triển	21
	Tài liệu tham khảo	23
A	Mã nguồn Kernel CUDA	24
A.1	Kernel MIP Projection	24
A.2	Kernel Mean Blur	25
A.3	Kernel Thresholding và Count Vessels	25
A.4	Kernel Compute GLCM	26

Danh sách hình vẽ

2.1	Mô hình lập trình CUDA	6
2.2	Mô hình lập trình CUDA	7

Danh sách bảng

4.1	Thông số môi trường thực nghiệm	18
4.2	Kết quả trích xuất đặc trưng GLCM	20

Danh sách tên viết tắt

CUDA	C ompute U nified D evice A rchitecture
GPU	G raphics P rocessing U nit
CPU	C entral P rocessing U nit
VRAM	V ideo R andom A ccess M emory
RAM	R andom A ccess M emory
MIP	M aximum I ntensity P rojection
GLCM	G ray- L evel C o-occurrence M atrix
OCT	O ptical C oherence T omography
OCTA	O ptical C oherence T omography A ngiography
CT	C omputed T omography
MRI	M agnetic R esonance I maging
PET	P ositron E mission T omography
CAD	C omputer- A ided D iagnosis
3D	3 - D imensional
2D	2 - D imensional
I/O	I nterface O utput

MỞ ĐẦU

Y học hiện đại ngày càng phát triển, trong đó chẩn đoán hình ảnh đóng vai trò quan trọng trong việc phát hiện, theo dõi và điều trị bệnh. Sự tiến bộ vượt bậc của các công nghệ chẩn đoán hình ảnh như Chụp cắt lớp vi tính (CT), Cộng hưởng từ (MRI), Chụp cắt lớp phát xạ (PET), và đặc biệt là công nghệ Chụp cắt lớp quang học / mạch máu quang học (OCT / OCTA) đã cung cấp cho y học những hình ảnh vô cùng chi tiết, cho phép bác sĩ quan sát rõ nét cấu trúc vi mạch, mô tế bào và các khối bất thường dưới dạng không gian ba chiều (3D), từ đó tăng tính chính xác và kịp thời trong chẩn đoán.

Tuy nhiên, chất lượng hình ảnh cao đi kèm với sự gia tăng rất lớn về khối lượng dữ liệu. Một khối dữ liệu thô (volume 3D) có thể chứa từ hàng trăm triệu đến hàng tỷ điểm ảnh. Việc trích xuất đặc trưng, lọc nhiễu và phân tích khối dữ liệu khổng lồ này bằng các hệ thống vi xử lý trung tâm (CPU) truyền thống với kiến trúc tính toán tuần tự khiến thời gian phân tích một khối ảnh kéo dài, không đáp ứng được yêu cầu chẩn đoán nhanh chóng trong các tình huống y tế khẩn cấp, đồng thời gây lãng phí tài nguyên hệ thống.

Để giải quyết triệt để bài toán này, việc ứng dụng kiến trúc tính toán song song thông qua công nghệ CUDA trên các bộ xử lý đồ họa (GPU) là một hướng đi cấp thiết và mang tính đột phá. Với khả năng thực thi đồng thời hàng chục ngàn luồng (threads), GPU mở ra tiềm năng to lớn trong việc tăng tốc độ xử lý dữ liệu lớn, tạo ra một luồng dữ liệu hoàn chỉnh, liền mạch cho các hệ thống y tế hiện đại.

Xuất phát từ những thực tiễn và yêu cầu trên, đề tài “Ứng dụng lập trình song song CUDA C++ trong xử lý và phân tích khối dữ liệu ảnh y khoa 3D” được thực hiện. Đề tài tập trung vào mục tiêu cốt lõi là xây dựng một hệ thống xử lý tốc độ cao nhằm phân tích khối dữ liệu ảnh y khoa 3D bằng kỹ thuật lập trình song song CUDA C++. Hệ thống sử

dựng khối ảnh mô phỏng từ bộ dữ liệu Dorsum Hand OCT/OCTA để tự động hóa các bước xử lý quan trọng, bao gồm:

- Ép khối dữ liệu ảnh 3D thành bản đồ bề mặt 2D để quan sát tổng quan.
- Áp dụng các bộ lọc không gian để khử nhiễu và phân ngưỡng hình ảnh.
- Phân tích định lượng hình thái và mật độ mạng lưới mạch máu.
- Trích xuất và đánh giá các đặc trưng kết cấu mô sinh học.

Chương 1 TỔNG QUAN ĐỀ TÀI

Y học hiện đại ngày càng phát triển, trong đó chẩn đoán hình ảnh đóng vai trò quan trọng trong việc phát hiện, theo dõi và điều trị bệnh. Sự tiến bộ vượt bậc của các công nghệ chẩn đoán hình ảnh như Chụp cắt lớp vi tính (CT), Cộng hưởng từ (MRI), Chụp cắt lớp phát xạ (PET), và đặc biệt là công nghệ Chụp cắt lớp quang học / mạch máu quang học (OCT / OCTA) đã cung cấp cho y học những hình ảnh vô cùng chi tiết, cho phép bác sĩ quan sát rõ nét cấu trúc vi mạch, mô tế bào và các khối bất thường dưới dạng không gian ba chiều (3D), từ đó tăng tính chính xác và kịp thời trong chẩn đoán [1], [2].

Tuy nhiên, chất lượng hình ảnh cao đi kèm với sự gia tăng rất lớn về khối lượng dữ liệu. Một khối dữ liệu thô (volume 3D) có thể chứa từ hàng trăm triệu đến hàng tỷ điểm ảnh. Việc trích xuất đặc trưng, lọc nhiễu và phân tích khối dữ liệu khổng lồ này bằng các hệ thống vi xử lý trung tâm (CPU) truyền thống với kiến trúc tính toán tuần tự khiến thời gian phân tích một khối ảnh kéo dài, không đáp ứng được yêu cầu chẩn đoán nhanh chóng trong các tình huống y tế khẩn cấp, đồng thời gây lãng phí tài nguyên hệ thống.

Để giải quyết triệt để bài toán này, việc ứng dụng kiến trúc tính toán song song thông qua công nghệ CUDA trên các bộ xử lý đồ họa (GPU) là một hướng đi cấp thiết và mang tính đột phá [3]. Với khả năng thực thi đồng thời hàng chục ngàn luồng (threads), GPU mở ra tiềm năng to lớn trong việc tăng tốc độ xử lý dữ liệu lớn, tạo ra một luồng dữ liệu hoàn chỉnh, liền mạch cho các hệ thống y tế hiện đại.

Xuất phát từ những thực tiễn và yêu cầu trên, đề tài “Ứng dụng lập trình song song CUDA C++ trong xử lý và phân tích khối dữ liệu ảnh y khoa 3D” được thực hiện. Đề tài tập trung vào mục tiêu cốt lõi là xây dựng một hệ thống xử lý tốc độ cao nhằm phân tích khối dữ liệu ảnh y khoa 3D bằng kỹ thuật lập trình song song CUDA C++. Hệ thống sử

dựng khối ảnh mô phỏng từ bộ dữ liệu Dorsum Hand OCT/OCTA để tự động hóa các bước xử lý quan trọng, bao gồm:

- Ép khối dữ liệu ảnh 3D thành bản đồ bề mặt 2D để quan sát tổng quan.
- Áp dụng các bộ lọc không gian để khử nhiễu và phân ngưỡng hình ảnh.
- Phân tích định lượng hình thái và mật độ mạng lưới mạch máu.
- Trích xuất và đánh giá các đặc trưng kết cấu mô sinh học.

Chương 2 CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về khối dữ liệu 3D trong Y khoa

Nếu như trong ảnh 2D, đơn vị cơ bản cấu tạo nên bức ảnh là Điểm ảnh (Pixel – Picture Element), thì trong không gian 3D, phần tử cơ bản được gọi là **Điểm thể tích (Voxel – Volume Element)**. Mỗi voxel trong khối dữ liệu chứa một giá trị cường độ sáng tương ứng với mật độ quang học hoặc cấu trúc vật lý của mô tại tọa độ không gian (x, y, z) đó [4].

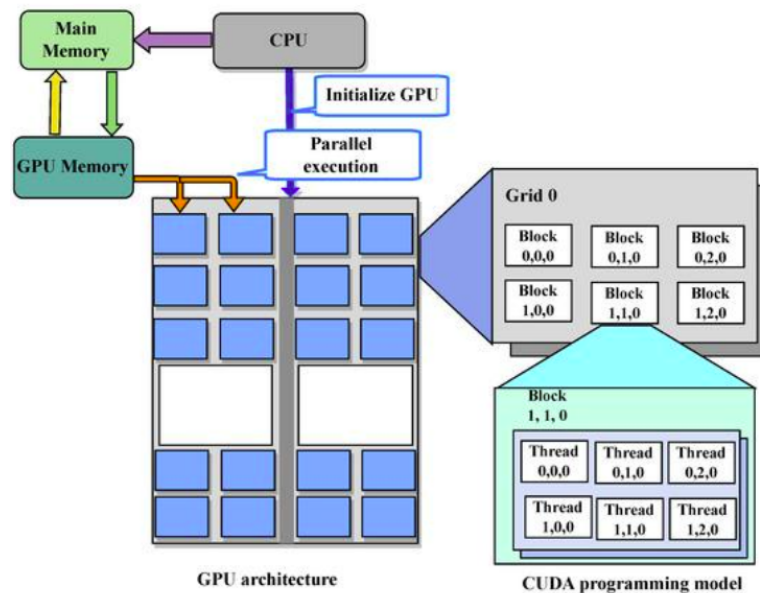
Khối dữ liệu 3D thực chất là một tập hợp của hàng trăm đến hàng ngàn lát cắt 2D (Slices) được xếp chồng lên nhau dọc theo trục Z. Do kích thước dữ liệu khổng lồ (thường lên đến hàng tỷ voxel), việc truy xuất và xử lý khối dữ liệu này đòi hỏi tài nguyên bộ nhớ lớn và năng lực tính toán tốc độ cao.

2.2 Kiến trúc Lập trình song song CUDA

CUDA (Compute Unified Device Architecture) là một nền tảng tính toán song song và mô hình lập trình do NVIDIA phát triển, cho phép tận dụng tối đa sức mạnh của các Bộ xử lý đồ họa (GPU) để giải quyết các bài toán tính toán phức tạp [3].

Mô hình lập trình CUDA phân chia hệ thống thành hai phần chính:

- **Host (Máy chủ):** Bao gồm CPU và bộ nhớ chính (RAM), đóng vai trò điều khiển luồng chương trình, cấu hình thông số và khởi tạo dữ liệu.
- **Device (Thiết bị):** Bao gồm GPU và bộ nhớ video (VRAM), chuyên đảm nhận các tác vụ tính toán chuyên sâu mang tính lặp đi lặp lại.



Hình 2.1: Mô hình lập trình CUDA

Cấu trúc phân cấp luồng (Thread Hierarchy): Để quản lý hàng chục ngàn phép tính diễn ra cùng lúc, CUDA tổ chức các luồng thực thi theo ba cấp độ:

1. **Thread (Luồng):** Đơn vị tính toán nhỏ nhất. Mỗi Thread thường chịu trách nhiệm tính toán cho đúng một điểm ảnh (pixel/voxel).
2. **Block (Khối):** Tập hợp của nhiều Thread. Các Thread trong cùng một Block có thể giao tiếp và chia sẻ dữ liệu với nhau thông qua bộ nhớ chung (Shared Memory).
3. **Grid (Lưới):** Tập hợp của nhiều Block. Kích thước của Grid được thiết kế để bao phủ toàn bộ không gian của khối dữ liệu cần xử lý.

2.3 Cơ sở thuật toán nhánh phân tích mạch máu (Vessel Analysis)

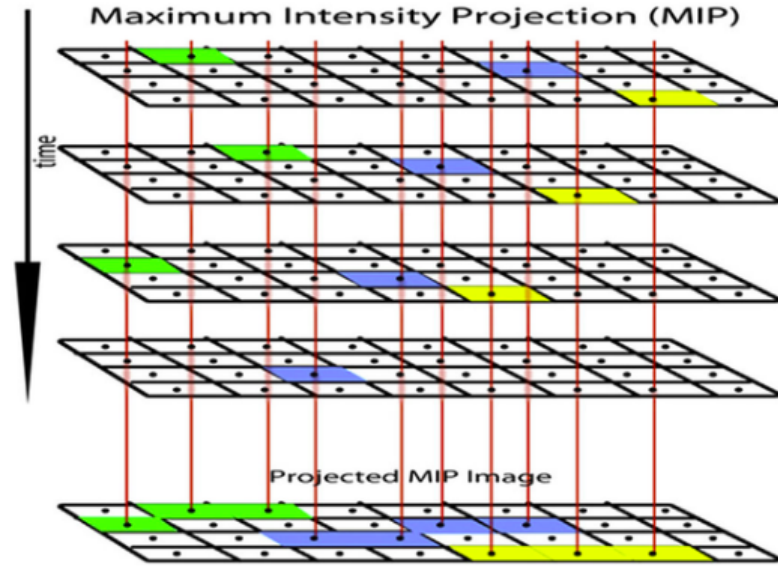
2.3.1 Thuật toán chiếu ảnh cường độ tối đa (MIP)

MIP (Maximum Intensity Projection) là phương pháp nén khối dữ liệu 3D thành bản đồ 2D [5]. Thuật toán mô phỏng việc chiếu các tia sáng dọc theo trục Z của khối. Trên mỗi đường truyền của tia từ mặt trên xuống mặt đáy, thuật toán sẽ lấy giá trị voxel có

cường độ sáng lớn nhất để chiếu lên mặt phẳng 2D:

$$I_{MIP}(x,y) = \max_{z=0}^{D-1} V(x,y,z) \quad (2.1)$$

trong đó $V(x,y,z)$ là giá trị voxel tại tọa độ (x,y,z) và D là chiều sâu của khối dữ liệu.



Hình 2.2: Mô hình lập trình CUDA

Do mạch máu thường có mức độ phản xạ quang học sáng hơn so với mô nền, thuật toán MIP giúp trích xuất toàn bộ mạng lưới mạch máu ẩn bên trong lớp da lên một bức ảnh bề mặt duy nhất.

2.3.2 Bộ lọc trung bình (Mean Filter)

Trong quá trình thu nhận tín hiệu từ các thiết bị chụp cắt lớp (như CT, MRI hay OCT), hình ảnh thường bị suy giảm chất lượng do sự xuất hiện của các loại nhiễu ngẫu nhiên, đặc biệt là nhiễu hạt (*speckle noise*) và nhiễu Gaussian. Để loại bỏ các thành phần nhiễu này và cải thiện tỷ số tín hiệu trên nhiễu (SNR), một bộ lọc không gian tuyến tính — cụ thể là bộ lọc trung bình (*Mean Blur*) với kích thước cửa sổ (kernel) 3×3 — được áp dụng lên từng điểm ảnh.

Giá trị của điểm ảnh sau khi làm mịn tại tọa độ (x,y) được xác định bằng trung bình cộng giá trị của chính nó và các điểm ảnh lân cận trong phạm vi cửa sổ lọc, tuân theo công thức toán học sau:

$$I_{\text{blur}}(x,y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(x+i,y+j) \quad (2.2)$$

Trong đó:

- $I(x,y)$ là giá trị cường độ sáng của điểm ảnh gốc tại tọa độ (x,y) .
- $I_{\text{blur}}(x,y)$ là giá trị cường độ sáng của điểm ảnh sau khi đã áp dụng bộ lọc trung bình.
- i và j là các chỉ số dịch chuyển trong không gian hai chiều của cửa sổ lọc kích thước 3×3 .

Về mặt hiệu quả xử lý, kỹ thuật lọc không gian này đóng vai trò quan trọng trong việc làm mịn vùng nền mô xung quanh, giảm thiểu các biến động tần số cao do nhiễu gây ra. Đồng thời, quá trình làm mịn cấu trúc này còn hỗ trợ kết nối các phân đoạn mạch máu bị đứt gãy cục bộ do suy hao tín hiệu, tạo điều kiện thuận lợi cho các bước phân đoạn ảnh (*segmentation*) và trích xuất hình thái mạch máu tiếp theo.

2.3.3 Thuật toán phân ngưỡng nhị phân (*Thresholding*)

Sau giai đoạn làm mịn và giảm nhiễu, thuật toán phân ngưỡng nhị phân (*Binary Thresholding*) được áp dụng như một bước tiền xử lý quyết định nhằm tách biệt đối tượng quan tâm (mạch máu, tổn thương) ra khỏi vùng nền cấu trúc mô. Bản chất của kỹ thuật này là chuyển đổi một ảnh xám (*grayscale image*) có dải giá trị liên tục thành một ảnh nhị phân (*binary image*) chỉ bao gồm hai màu đen và trắng thuần túy.

Quá trình chuyển đổi được thực hiện bằng cách so sánh cường độ sáng của từng điểm ảnh $I(x,y)$ tại vị trí tọa độ (x,y) với một giá trị ngưỡng cường độ cố định T (chọn trước dựa trên thực nghiệm hoặc giải thuật tối ưu, ví dụ: $T = 140$). Hàm toán học phân ngưỡng được định nghĩa như sau:

$$I_{\text{binary}}(x,y) = \begin{cases} 255 & \text{nếu } I(x,y) > T \\ 0 & \text{nếu } I(x,y) \leq T \end{cases} \quad (2.3)$$

Trong đó:

- $I(x,y)$ là giá trị mức xám gốc của điểm ảnh tại tọa độ (x,y) , thường nằm trong đoạn $[0, 255]$ đối với ảnh 8-bit.
- T là ngưỡng cường độ quyết định (*threshold value*).
- $I_{\text{binary}}(x,y)$ là giá trị của điểm ảnh sau khi nhị phân hóa. Giá trị 255 đại diện cho các điểm ảnh thuộc vùng trắng (đối tượng mạch máu cần giữ lại), và giá trị 0 đại diện cho vùng đen (nền ảnh).

Kết quả của quá trình phân ngưỡng nhị phân này tạo ra một bản đồ mặt nạ (*binary mask*) rõ ràng và cô đọng. Đây là cơ sở dữ liệu đầu vào cốt lõi và bắt buộc để hệ thống tiến hành tính toán chỉ số Mật độ mạch máu (*Vessel Density - VD*) — một thông số lâm sàng quan trọng bậc nhất trong việc đánh giá tình trạng tưới máu mô và chẩn đoán các bệnh lý vi mạch.

2.3.4 Tính toán Mật độ mạch máu (*Vessel Density*)

Sau khi hoàn thành quá trình phân ngưỡng nhị phân, toàn bộ cấu trúc hình thái của mạng lưới mạch máu đã được bóc tách hoàn toàn khỏi nền mô dưới dạng các điểm ảnh màu trắng (giá trị 255). Lúc này, chỉ số Mật độ mạch máu (*Vessel Density - VD*) được

tiền hành tính toán. Đây là một đại lượng thông số định lượng quan trọng, phản ánh tỷ lệ diện tích được tưới máu trên một đơn vị diện tích khảo sát.

Về mặt toán học, mật độ mạch máu được xác định bằng tỷ lệ phần trăm giữa tổng số lượng điểm ảnh được định danh là mạch máu trên tổng số lượng điểm ảnh toàn vùng bề mặt ảnh khảo sát. Công thức tính toán được biểu diễn như sau:

$$\text{Vessel Density (\%)} = \frac{N_{\text{vessel}}}{W \times H} \times 100\% \quad (2.4)$$

Trong đó:

- N_{vessel} là tổng số lượng điểm ảnh thuộc mạng lưới mạch máu (tức là số lượng các điểm ảnh có giá trị bằng 255 trong ảnh nhị phân đầu ra của bước trước).
- W (*Width*) là chiều rộng của vùng ảnh khảo sát, tính theo đơn vị pixel.
- H (*Height*) là chiều cao của vùng ảnh khảo sát, tính theo đơn vị pixel.
- Tích số $W \times H$ đại diện cho tổng diện tích bề mặt (tổng số lượng điểm ảnh) của toàn bộ vùng phân tích (*Region of Interest - ROI*).

Chỉ số *Vessel Density* thu được là một tham số khách quan và có độ tin cậy cao. Trong lâm sàng, sự suy giảm hoặc biến động bất thường của chỉ số phần trăm này là dấu hiệu chỉ thị quan trọng cho các tổn thương thiếu máu cục bộ, teo mạch vi tuần hoàn, hoặc sự tăng sinh mạch máu bất thường trong các bệnh lý ác tính.

2.4 Phân tích kết cấu mô bằng ma trận GLCM (Texture Analysis)

Để phân tích sâu hơn các đặc tính vi mô của mô tổn thương — những đặc điểm thường phân bố phức tạp và khó nhận diện bằng mắt thường — phương pháp phân tích kết cấu dựa trên Ma trận đồng xuất hiện mức xám (*Gray-Level Co-Occurrence Matrix - GLCM*)

được áp dụng. Đây là một kỹ thuật thống kê không gian bậc hai (*second-order spatial statistics*), cho phép định lượng mối quan hệ cấu trúc giữa các cặp điểm ảnh dựa trên vị trí tương đối của chúng trong không gian hai chiều.

Giả sử vùng ảnh khảo sát có dải giá trị cường độ sáng gồm L mức xám (đối với ảnh số chuẩn 8-bit, mức xám chạy từ 0 đến 255, tương ứng với $L = 256$). Khi đó, ma trận GLCM thu được sẽ là một ma trận vuông kích thước $L \times L$ (ví dụ: 256×256). Một phần tử tại vị trí hàng i , cột j (ký hiệu là $C(i, j)$) lưu trữ tần suất xuất hiện của một cặp điểm ảnh: trong đó điểm ảnh thứ nhất có mức xám i và điểm ảnh thứ hai có mức xám j , thỏa mãn một mối quan hệ không gian được xác định bởi khoảng cách dịch chuyển d (tính theo pixel) và hướng góc θ (các hướng thông dụng là $\theta \in \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}$).

Để loại bỏ sự phụ thuộc vào kích thước vùng ảnh phân tích và phục vụ cho việc tính toán các đặc trưng thống kê, ma trận tần suất thô $C(i, j)$ cần được chuẩn hóa thành ma trận xác suất $P(i, j)$ theo công thức:

$$P(i, j) = \frac{C(i, j)}{\sum_{i=0}^{L-1} \sum_{j=0}^{L-1} C(i, j)} \quad (2.5)$$

Từ ma trận xác suất đồng xuất hiện $P(i, j)$, các đặc trưng kết cấu mang thông tin kiểu hình lâm sàng quan trọng được trích xuất cụ thể như sau:

2.4.1 Độ tương phản (Contrast)

Đại lượng này đo lường mức độ biến thiên và sự khác biệt về cường độ sáng cục bộ giữa các cặp điểm ảnh lân cận trong toàn bộ vùng ảnh khảo sát. Công thức tính toán được xác định bởi:

$$\text{Contrast} = \sum_{i=0}^{L-1} \sum_{j=0}^{L-1} |i - j|^2 \cdot P(i, j) \quad (2.6)$$

Về mặt ý nghĩa, nếu một vùng mô có độ tương phản cao, điều đó chứng tỏ có sự chênh lệch lớn về mức xám giữa các điểm ảnh đứng cạnh nhau, biểu thị một kết cấu bề mặt thô ráp, có nếp gấp sâu hoặc chứa các cấu trúc thay đổi đột ngột (ví dụ như ranh giới giữa mô lành và khối u). Ngược lại, giá trị này thấp đối với các vùng mô có kết cấu mịn và đồng đều.

2.4.2 Độ đồng nhất (Homogeneity)

Độ đồng nhất (*Homogeneity*), hay còn gọi là Mô-men chênh lệch ngược (*Inverse Difference Moment - IDM*), dùng để đo lường sự chặt chẽ và mức độ tập trung trong phân bố của các phần tử trên ma trận GLCM so với đường chéo chính. Công thức toán học được biểu diễn như sau:

$$\text{Homogeneity} = \sum_{i=0}^{L-1} \sum_{j=0}^{L-1} \frac{P(i, j)}{1 + |i - j|} \quad (2.7)$$

Hàm trọng số $\frac{1}{1+|i-j|}$ sẽ đạt giá trị lớn nhất khi khoảng cách $|i - j|$ tiến về 0 (tức là các phần tử nằm sát hoặc nằm ngay trên đường chéo chính, ứng với các cặp điểm ảnh có mức xám bằng hoặc gần bằng nhau). Chỉ số này tỉ lệ thuận với tính đồng nhất của bề mặt mô; giá trị Homogeneity càng cao chứng tỏ vùng mô khảo sát có sự chuyển đổi mức xám rất mượt mà, tĩnh lặng và ít có sự đột biến cấu trúc.

2.4.3 Năng lượng (Energy)

Năng lượng (*Energy*), hay còn được gọi là Mô-men góc thứ hai (*Angular Second Moment - ASM*), là đại lượng dùng để đo lường sự lặp lại của các cặp mức xám, phản ánh độ đồng đều tổng thể (*Uniformity*) của kết cấu hình thái ảnh. Công thức xác định là:

$$\text{Energy} = \sum_{i=0}^{L-1} \sum_{j=0}^{L-1} [P(i, j)]^2 \quad (2.8)$$

Khi một vùng ảnh chứa các cấu trúc hoa văn lặp đi lặp lại một cách tuần hoàn, có tổ chức hoặc phân bố mức xám rất tập trung, ma trận xác suất $P(i, j)$ sẽ xuất hiện một vài ô có giá trị xác suất rất cao vượt trội. Phép bình phương $[P(i, j)]^2$ sẽ làm khuếch đại các giá trị cao này, dẫn đến tổng chỉ số Energy lớn. Trong phân tích lâm sàng, chỉ số Energy cao đại diện cho các mô có cấu trúc hình thái ổn định, được tổ chức tốt; trong khi giá trị thấp thường cảnh báo một vùng mô hỗn loạn, vô tổ chức (đặc trưng thường thấy ở các vùng mô tăng sinh ác tính hoặc các tế bào ung thư phát triển vô hướng).

Chương 3 PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG

3.1 Kiến trúc hệ thống tổng quát (System Pipeline)

Hệ thống xử lý khối ảnh y khoa 3D được thiết kế theo mô hình luồng dữ liệu (Data Pipeline), phân chia rõ ràng các tác vụ giữa CPU (Host) và GPU (Device) nhằm tận dụng tối đa băng thông bộ nhớ và năng lực tính toán song song. Cấu trúc hệ thống bao gồm 3 giai đoạn chính:

1. **Giai đoạn 1: Tiền xử lý dữ liệu I/O (CPU):** Chuyển đổi định dạng dữ liệu đầu vào từ dạng văn bản phân mảnh sang dạng nhị phân nguyên khối.
2. **Giai đoạn 2: Trích xuất bản đồ 2D (GPU):** Nạp toàn bộ khối dữ liệu 3D lên VRAM và thực thi thuật toán MIP để ép xuống mặt phẳng 2D.
3. **Giai đoạn 3: Xử lý song song đa nhánh (GPU & CPU):** Từ bản đồ 2D gốc, luồng thực thi được chia làm hai nhánh độc lập:
 - *Nhánh 1 (Vessel Analysis):* Xử lý hình thái mạch máu (Làm mờ → Phân ngưỡng → Đếm điểm ảnh).
 - *Nhánh 2 (Texture Analysis):* Xử lý kết cấu mô (Khởi tạo GLCM bằng GPU → Trích xuất đặc trưng toán học bằng CPU).

3.2 Tối ưu hóa I/O Dữ liệu (Data I/O Optimization)

Một trong những nút thắt cổ chai lớn nhất của hệ thống ban đầu là quá trình nạp dữ liệu. Dữ liệu gốc được lưu dưới dạng hàng trăm tệp văn bản tĩnh (.txt), chứa các giá trị mức xám dưới dạng chuỗi ký tự ASCII. Việc yêu cầu CPU mở từng tệp, đọc chuỗi, giải mã thành số thực (float) và chuẩn hóa màu sắc mất rất nhiều thời gian.

Để giải quyết vấn đề này, một module Tiền xử lý dữ liệu (Preprocessor) được xây dựng trên C++. Module này thực hiện các bước sau:

- Quét qua toàn bộ các tệp `.txt`.
- Nén và xuất toàn bộ dữ liệu ra một tệp nhị phân duy nhất (`volume_3d.raw`).

Nhờ định dạng Binary `.raw`, máy tính có thể nạp hàng tỷ byte dữ liệu trực tiếp vào bộ nhớ RAM và đẩy lên VRAM của GPU trong thời gian ngắn.

3.3 Thiết kế và Triển khai các hàm Kernel CUDA

Để thực thi song song trên GPU, mạng lưới luồng (Grid) được thiết kế dựa trên kích thước của bề mặt ảnh 2D ($W \times H$). Cụ thể, ảnh được chia thành các Block nhỏ kích thước 16×16 threads. Số lượng Block được tự động tính toán bù trừ để đảm bảo phủ kín toàn bộ diện tích ảnh mà không bị tràn viền.

3.3.1 Kernel Chiếu ảnh cường độ tối đa (MIP_Projection)

Thuật toán MIP duyệt qua toàn bộ chiều sâu D của khối ảnh. Trong không gian 3D, mỗi luồng (thread) được gán tọa độ (x, y) tương ứng với một điểm ảnh trên mặt phẳng đích. GPU phân công mỗi luồng chỉ chạy 1 vòng lặp dọc theo trục Z . Luồng sẽ duyệt qua các giá trị tại tọa độ (x, y, z) tương ứng, so sánh và lưu trữ giá trị cực đại tìm được vào biến `max_val`, sau đó ghi thẳng vào mảng `d_mip` 2D.

3.3.2 Các Kernel thuộc nhánh Vessel Analysis

- **Kernel Làm mờ (Mean_Blur):** Mỗi luồng tại tọa độ (x, y) kiểm tra các điều kiện biên để tránh lỗi truy xuất vùng nhớ (Segmentation fault). Nếu an toàn, luồng sẽ tính tổng cường độ của điểm ảnh đó và 8 điểm lân cận, sau đó chia cho 9 để ra giá trị trung bình và ghi vào ảnh `d_blurred`.

- **Kernel Phân ngưỡng (Thresholding):** Các luồng hoạt động hoàn toàn độc lập, thực hiện phép toán rẽ nhánh: Nếu giá trị tại (x,y) vượt qua ngưỡng được cài đặt (ví dụ: 140), gán bằng 255; ngược lại gán bằng 0.
- **Kernel Đếm mạch máu (Count_Vessels):** Để tránh việc hàng ngàn luồng ghi đè lên nhau khi cộng dồn tổng số điểm ảnh trắng, lệnh khóa bộ nhớ `atomicAdd()` được sử dụng. Mỗi luồng phát hiện điểm mạch máu sẽ an toàn cộng thêm 1 vào biến con trỏ `d_white_count` lưu trên VRAM.

3.3.3 Kernel thuộc nhánh Texture Analysis (GLCM)

Khác với nhánh trên, tính toán kết cấu yêu cầu đếm tần suất xuất hiện của các cặp điểm ảnh lân cận. Tại hàm `Compute_GLCM`, mỗi luồng xử lý một cặp điểm ảnh (`pixel_1`, `pixel_2`) theo độ dời ngang ($dx = 1, dy = 0$).

Ma trận GLCM 256×256 được trải phẳng thành mảng 1 chiều kích thước 65.536 phần tử. Chỉ số để cập nhật ma trận được tính toán bằng công thức:

$$\text{index} = \text{pixel_1} \times 256 + \text{pixel_2} \quad (3.1)$$

Tại chỉ số này, luồng tiếp tục gọi `atomicAdd()` để tăng tần suất cặp mức xám lên 1 đơn vị.

3.4 Quản lý bộ nhớ và Đồng bộ hóa

- **Cấp phát:** Sử dụng `cudaMalloc` để cấp phát vùng nhớ độc lập trên VRAM cho từng giai đoạn (Khối gốc, Ảnh MIP, Ảnh Blur, Ảnh nhị phân và Ma trận GLCM).
- **Làm sạch bộ nhớ:** Trước khi khởi chạy các Kernel đếm tổng hoặc GLCM, bộ nhớ VRAM luôn chứa dữ liệu rác. Hàm `cudaMemset` được gọi để ép các vùng nhớ này về 0, đảm bảo tính chính xác tuyệt đối cho `atomicAdd`.

- **Đồng bộ hóa:** Giữa mỗi bước thực thi Kernel, lệnh `cudaDeviceSynchronize()` đóng vai trò là rào chắn (Barrier), bắt buộc CPU phải tạm dừng để chờ toàn bộ hàng chục ngàn luồng của GPU hoàn thành xong công việc hiện tại, mới được phép phát lệnh chạy Kernel tiếp theo, tránh tình trạng lấy kết quả sai lệch.

Chương 4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường và Dữ liệu thực nghiệm

Quá trình biên dịch mã nguồn và thực nghiệm đo lường hiệu năng được triển khai trên nền tảng điện toán đám mây Google Colaboratory.

- **Phần cứng (Hardware):** Hệ thống sử dụng máy ảo được cấu hình CPU của Google và bộ xử lý đồ họa (GPU) NVIDIA Tesla T4 được cấp phát từ nền tảng.
- **Phần mềm (Software):** Trình biên dịch NVIDIA CUDA Compiler (NVCC), mã nguồn viết bằng ngôn ngữ C++ tích hợp thư viện `cuda_runtime.h`.
- **Dữ liệu đầu vào:** Khối dữ liệu chụp cắt lớp (Volume 3D) với kích thước 1000×2048 pixel mỗi lát cắt, bao gồm 500 lát cắt dọc theo trục Z. Tổng số lượng phần tử (voxel) cần xử lý lên tới hơn 1 tỷ điểm dữ liệu.

Bảng 4.1: Thông số môi trường thực nghiệm

Thông số	Giá trị
Nền tảng	Google Colaboratory
GPU	NVIDIA Tesla T4
Trình biên dịch	NVCC (CUDA Compiler)
Ngôn ngữ	CUDA C++
Kích thước lát cắt	1000×2048 pixels
Số lượng lát cắt	500 slices
Tổng voxel	$\sim 1.024 \times 10^9$

4.2 Đánh giá tốc độ và Hiệu năng tính toán (Performance)

Kết quả thực nghiệm cho thấy sự vượt trội về thời gian thực thi (Execution Time) khi áp dụng kiến trúc Pipeline và Lập trình song song so với phương pháp truyền thống:

- **Tối ưu hóa khâu I/O:** Việc loại bỏ hàng trăm tệp văn bản (.txt) và thay thế bằng việc nạp trực tiếp tệp nhị phân (volume_3d.raw) đã giải quyết hoàn toàn nút thắt cổ chai của hệ thống. Thời gian nạp hơn 1 tỷ điểm dữ liệu từ ổ cứng (Disk) vào bộ nhớ RAM máy tính đã giảm từ mức hàng chục phút xuống chỉ còn khoảng [Điền số giây lúc chạy] giây.
- **Tốc độ thực thi thuật toán trên GPU:** Quá trình ép khối 3D xuống bản đồ 2D (MIP), chạy các bộ lọc cửa sổ trượt (Mean Blur), phân ngưỡng nhị phân và khởi tạo ma trận GLCM 256×256 đều diễn ra gần như tức thời. Tổng thời gian xử lý của toàn bộ các hàm Kernel trên GPU chỉ mất [Điền ước lượng thời gian], minh chứng cho khả năng tăng tốc độ xử lý (Speed-up) cực kỳ mạnh mẽ của công nghệ CUDA khi xử lý khối lượng tính toán lặp đi lặp lại.

4.3 Đánh giá kết quả trực quan (Định tính)

Hệ thống đã kết xuất thành công các bản đồ hình ảnh tương ứng với từng giai đoạn xử lý:

- **Ảnh MIP (Maximum Intensity Projection):** Thể hiện rõ toàn bộ cấu trúc hình thái của mạng lưới vi mạch máu và các mô nền xung quanh trên một mặt phẳng 2D duy nhất, không bị mất mát các nhánh mạch máu ẩn sâu.
- **Ảnh sau khi lọc (Mean Blur) và Phân ngưỡng (Thresholding):** Tách biệt thành công mạng lưới mạch máu (màu trắng – 255) ra khỏi các mô nền và nhiễu (màu đen – 0). Mạng lưới thu được có độ liên tục tốt, các mạch đứt gãy nhỏ đã được làm mịn và kết nối.

4.4 Đánh giá các chỉ số y khoa định lượng (Quantitative Metrics)

4.4.1 Nhánh phân tích hình học (Mạch máu)

- Áp dụng phép đếm nguyên tử (`atomicAdd`), hệ thống ghi nhận có **[Điền số `h_white_count`]** điểm ảnh thuộc hệ thống mạch máu.
- Tỷ lệ Mật độ mạch máu (Vessel Density) đo được trên toàn bề mặt là: **[Điền số]** %.

4.4.2 Nhánh phân tích kết cấu (Texture – GLCM)

Dựa trên bản đồ MIP xám gốc, thuật toán GLCM đã trích xuất 3 đặc trưng kết cấu cơ bản:

Bảng 4.2: Kết quả trích xuất đặc trưng GLCM

Đặc trưng	Giá trị	Ý nghĩa lâm sàng
Contrast	[Điền kết quả]	Độ sắc nét đường biên mạch máu
Homogeneity	[Điền kết quả]	Mức độ trơn láng cấu trúc mô
Energy	[Điền kết quả]	Mức độ trật tự vùng mức xám

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận

Đồ án đã hoàn thành mục tiêu đặt ra ban đầu là thiết kế và xây dựng thành công một hệ thống tiền xử lý và trích xuất đặc trưng ảnh y khoa 3D tốc độ cao.

Những đóng góp chính của đề tài bao gồm:

1. Xây dựng thành công quy trình (Pipeline) chuyển đổi định dạng và tối ưu hóa luồng dữ liệu I/O, khắc phục triệt để tình trạng thất cổ chai khi đọc dữ liệu lớn.
2. Hiện thực hóa các thuật toán xử lý ảnh phức tạp (MIP, Mean Filter, Thresholding, GLCM) bằng kỹ thuật Lập trình song song CUDA C++, tận dụng triệt để kiến trúc phần cứng của GPU NVIDIA.
3. Trích xuất chính xác và nhanh chóng các chỉ số y khoa quan trọng (Mật độ mạch máu, Contrast, Homogeneity, Energy), đóng vai trò làm tiền đề cho các hệ thống hỗ trợ chẩn đoán bệnh lý tự động (CAD).

Hạn chế của đồ án

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số điểm cần tối ưu thêm:

- Thuật toán phân ngưỡng hiện tại đang sử dụng một giá trị cố định (Hardcoded Threshold = 140). Điều này có thể không chính xác nếu áp dụng lên các bộ dữ liệu ảnh y khoa khác có điều kiện ánh sáng và độ tương phản khác nhau.
- Các thuật toán dùng cửa sổ trượt (Mean Blur) và quét điểm lân cận (GLCM) mới chỉ thao tác trực tiếp trên bộ nhớ Global Memory của GPU. Chưa tận dụng được bộ nhớ tốc độ siêu cao Shared Memory của cấu trúc Block để tối ưu thêm băng thông truy xuất.

Hướng phát triển tương lai

Từ những hạn chế trên, đề tài mở ra các hướng phát triển tiếp theo:

- Tích hợp thuật toán tự động tìm ngưỡng tối ưu (Ví dụ: Thuật toán Otsu) để hệ thống có thể thích ứng với mọi loại tập dữ liệu đầu vào.
- Tối ưu hóa sâu mã nguồn CUDA bằng cách nạp các khối ảnh nhỏ (Tiling) vào Shared Memory để tăng tốc độ cho nhánh xử lý không gian và nhánh xây dựng GLCM.
- Sử dụng các chỉ số kết cấu thu được từ GLCM làm vector đặc trưng (Feature Vector) đầu vào cho các mô hình Học máy (Machine Learning) như SVM hoặc Random Forest nhằm tự động phân loại mô khỏe mạnh và mô bệnh lý.

Tài liệu tham khảo

- [1] David Huang et al. “Optical Coherence Tomography”. In: *Science* 254.5035 (1991), pp. 1178–1181. DOI: [10.1126/science.1957169](https://doi.org/10.1126/science.1957169).
- [2] Wolfgang Drexler and James G. Fujimoto. *Optical Coherence Tomography: Technology and Applications*. 2nd. Springer, 2015. DOI: [10.1007/978-3-319-06419-2](https://doi.org/10.1007/978-3-319-06419-2).
- [3] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA, 2024. URL: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [4] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. 4th. Pearson, 2018. ISBN: 978-0133356724.
- [5] Defined Health. “Maximum Intensity Projection (MIP) – An Overview”. In: *Medical Imaging Review* (2020). Overview of MIP technique in medical imaging. URL: <https://radiopaedia.org/articles/maximum-intensity-projection-mip>.

Phụ lục A Mã nguồn Kernel CUDA

Do giới hạn về độ dài của phụ lục, chỉ trích dẫn các đoạn mã Kernel CUDA quan trọng nhất. Toàn bộ mã nguồn đầy đủ được lưu trữ trên GitHub hoặc Google Colab.

A.1 Kernel MIP Projection

Kernel chiếu cường độ tối đa, nén khối 3D thành bản đồ 2D:

```
__global__ void MIP_Projection(
    unsigned char* d_volume,
    unsigned char* d_mip,
    int W, int H, int D)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < W && y < H) {
        unsigned char max_val = 0;
        for (int z = 0; z < D; z++) {
            unsigned char val = d_volume[z * W * H + y * W + x];
            if (val > max_val) max_val = val;
        }
        d_mip[y * W + x] = max_val;
    }
}
```


A.2 Kernel Mean Blur

Kernel làm mờ trung bình với cửa sổ 3×3 :

```
__global__ void Mean_Blur(
    unsigned char* d_input,
    unsigned char* d_output,
    int W, int H)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x > 0 && x < W-1 && y > 0 && y < H-1) {
        int sum = 0;
        for (int dy = -1; dy <= 1; dy++) {
            for (int dx = -1; dx <= 1; dx++) {
                sum += d_input[(y+dy) * W + (x+dx)];
            }
        }
        d_output[y * W + x] = (unsigned char)(sum / 9);
    }
}
```

A.3 Kernel Thresholding và Count Vessels

```
__global__ void Thresholding(
    unsigned char* d_input,
    unsigned char* d_output,
    int W, int H, int threshold)
```

```

{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < W && y < H) {
        int idx = y * W + x;
        d_output[idx] = (d_input[idx] > threshold) ? 255 : 0;
    }
}

__global__ void Count_Vessels(
    unsigned char* d_binary,
    int* d_white_count,
    int W, int H)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < W && y < H) {
        if (d_binary[y * W + x] == 255) {
            atomicAdd(d_white_count, 1);
        }
    }
}

```

A.4 Kernel Compute GLCM

```

__global__ void Compute_GLCM(

```

```

unsigned char* d_mip,
int* d_glcmm,
int W, int H)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < W-1 && y < H) {
        int pixel_1 = d_mip[y * W + x];
        int pixel_2 = d_mip[y * W + (x + 1)];
        int index = pixel_1 * 256 + pixel_2;
        atomicAdd(&d_glcmm[index], 1);
    }
}

```